FAST – National University of Computer and Emerging Sciences

Theory of Automata

Assignment # 2

Deadline: 31 October 2025 Instructions:

1. Attempt all questions.
2. Questions must be attempted on paper (A4 sheets or loose sheets)
3. 3. Submit the assignment on time. Only physical submissions are accepted.

Part A – Kleene's Theorem: (2x3 + 2x3 = 12 marks)

Questions 1:

Kleene's Theorem Part 3 states that for every regular expression, there exists an equivalent finite automaton.
Kleene's Theorem Part 2 states that for every finite automaton, there exists an equivalent regular expression.

Task:
a) Using Kleene's Theorem Part 3, construct an NFA for the regular expression (a+b)*ab.
b) Using Kleene's Theorem Part 2, derive the equivalent regular expression for the following finite automaton:
States = {q0, q1, q2}
Alphabet = {a, b}
Start State = q0
Final State = q2
Transition Table:
q0 --a--> q1
q0 --b--> q0
q1 --a--> q1
q1 --b--> q2
q2 --a--> q2
q2 --b--> q2
Show all construction and derivation steps clearly.

Questions 2:

a) Construct a regular expression corresponding to the finite automaton given below using Kleene's Theorem Part 2.
States = {S, A, B}
Alphabet = {0, 1}
Start State = S
Final State = B
Transitions:

S --0--> S
S --1--> A
A --0--> B
A --1--> S
B --0--> A
B --1--> B

b) Convert the regular expression (0+1)*11 into an equivalent NFA using Kleene's Theorem Part 3.
Show all construction steps clearly.

Part B - Grammar, FA, PDA, CFG and Derivations:(20+9+9+4+9+10+10 = 71 marks)

Questions 1:

Design Regular Grammar and PDA for given Regular Language.

1. 01* (0+1)*
2. 01(0+1)*
3. Letter followed by digit
4. Start and end with a
5. Generate all even integer upto 998
6. Alternate letters
7. a^2n
8. (ab)^n
9. { 012, 120, 201} generate these strings
10. Construct a grammar for the set of all strings over {0,1} consisting of equal number of 0's and 1's.

Questions 2:

Draw the equivalent Finite Automaton (FA) for each grammar.

a. $S \rightarrow aA \mid bB$
$A \rightarrow aA \mid bS \mid \varepsilon$
$B \rightarrow bB \mid aS$

b. $S \rightarrow 0S \mid 1A$
$A \rightarrow 0A \mid 1$

c. $S \rightarrow 0A \mid 1B$
$A \rightarrow 1S \mid 0A$
$B \rightarrow 0S \mid 1B$

Questions 3:

For each FA (drawn or described), write the corresponding Right Linear Grammar or Context-Free Grammar.

a. DFA that accepts strings ending with "01".
b. NFA for language L = { w | w starts with a and ends with b }.
c. NFA that accepts all strings containing substring "aba".

Questions 4:

Convert each grammar as asked**.**

a. Convert the following **LLG → RLG**
$S \rightarrow Aa \mid Ab$
$A \rightarrow Ba \mid Bb \mid a$

b. Convert the following **RLG → LLG**
$S \rightarrow aA \mid bB$
$A \rightarrow a \mid aS \mid b$
$B \rightarrow b \mid bS \mid a$

Questions 5:

For each grammar:
(a) Derive the given string using substitution (step-by-step).
(b) Show Leftmost and Rightmost derivations.
(c) Draw the Parse Tree.

1. $S \rightarrow aSb \mid ab$
String: **aabb**

2. $S \rightarrow aS \mid Sb \mid ab$
String: **aaabb**

3. $S \rightarrow 0S1 \mid 01$
String: **000111**

Questions 6:

Design Context Free Grammar and PDA for given Context Free Language.

I. To design a context free grammar which can produce balance parenthesis consider in alphabet set we have { (,)}

II. Design a CFG and PDA for even and odd palindrom

   a. Even palindrome 001 100

   b. Odd palindrome 00100

III. Design a CFG and PDA for language $\{01(1100)^n\ 110\ (10)^n \mid n>=0\}$

IV. Design a CFG and PDA for language $\{ o^n\ 1^m \mid n != m\}$

V. Design a CFG and PDA for language $\{ (0^n\ 1^n\ 2^m\ 3^m \mid n>=1, m>=1)$ or $(0^n\ 1^m\ 2^m\ 3^n \mid n>=1, m>=1) \}$

VI. Design a CFG and PDA for language $\{ (0^i\ 1^j\ 2^k \mid k <= i, k <= j)$

VII. Design a CFG and PDA for language $\{ (0^i\ 1^j\ 2^{k-} \mid i==j,\ j == k\}$

VIII. Design a CFG and PDA for language $\{ (0^n\ 1^m\ 2^{2m-} \mid n,m>=0\}$

IX. Design a CFG and PDA for language $\{ (0^n\ 1^{m\ -} \mid n<= m+3 \}$

X. Design a CFG and PDA for language $\{ (0^n\ 1^{m\ -} \mid 2^n <= m <= 3^n\}$

Questions 7:

Scenario-Based Question:
You are developing a **syntax validation module** for a simple **programming language interpreter** that checks **nested structures** like parentheses, curly braces, and square brackets.
To model this system, you are required to **design a Pushdown Automaton (PDA)** that verifies the correctness of **balanced and properly nested symbols** in program code.

Description of the Problem:
A valid program string contains balanced pairs of (), {}, and [] such that:

1. Every opening symbol has a corresponding closing symbol of the same type.
2. Symbols are **properly nested** — e.g., { [( )] } is valid but { ( ]) } is not.
3. The input alphabet Σ = { (,), {, }, [,] }.

Tasks:
a) Define the **formal 7-tuple (Q, Σ, Γ, δ, q_0, Z_0, F)** of the PDA that accepts valid nested symbol strings.
b) Explain the **working principle** of your PDA (push and pop behavior).
c) Draw the **state transition diagram** of your PDA.
d) Trace the **step-by-step stack operations** for the input string { [( )] }.
e) Write an **equivalent Context-Free Grammar (CFG)** that generates the same language accepted by your PDA.

Part C – Grammar Simplification: (1x5 = 5 marks)

Consider the following CFG for non-empty language:
S→aSa|bSb|a|b

A→B | a

B →C | b

a. Remove useless productions from the given CFG, if there are any.

b. Check for the ambiguity in the given CFG for the string 'abba'.

c. Convert the given CFG into PDA.

d. Give the instantaneous description of resultant PDA.

e. Trace the input string 'baab' on stack using the resultant PDA.

Part D – Minimization of DFA: (6+6= 12 marks)

Questions 1:

Given the following DFA:

States = {A, B, C, D, E}
Alphabet = {0, 1}
Start State = A
Final State(s) = {C, E}
Transition Table:
A --0--> B
A --1--> C
B --0--> A
B --1--> D
C --0--> E
C --1--> C
D --0--> E
D --1--> C
E --0--> E
E --1--> C

Task:
a) Apply the DFA minimization algorithm to minimize the DFA.
b) Show all intermediate steps clearly.
c) Draw the minimized DFA.

Questions 2:

Consider a DFA defined as follows:
States = {q0, q1, q2, q3}
Alphabet = {a, b}
Start State = q0
Final State(s) = {q2, q3}
Transition Table:

q0 --a--> q1
q0 --b--> q0
q1 --a--> q2
q1 --b--> q1
q2 --a--> q3
q2 --b--> q2
q3 --a--> q3
q3 --b--> q3

Task:
a) Identify all equivalent states.
b) Construct the minimized DFA step by step.
c) Write the new transition table of the minimized DFA.